

Vibe Coding 图像颜色空间、图像插值

1 Prompt

1. 实现 RGB, HSV 等颜色空间相关的实现, 要求输入图像, 可以输出不同颜色空间中每个通道的图像
2. 在上述功能基础上, 要求实现图像插值算法: 最近邻、双线性插值 (输入图像, 选择图像操作: 放大、缩小、旋转拉伸等, 输出插值后的图像)
3. 在 test_image 这张图像上实现上述两个要求
4. 最后将上述功能做成一个网页, 集成所有功能, 并且提供可选择选项

2 核心代码

```
def rgb_to_hsv(rgb_image):
    rgb = rgb_image.astype(np.float32) / 255.0
    r, g, b = rgb[:, :, 0], rgb[:, :, 1], rgb[:, :, 2]

    max_val = np.maximum(np.maximum(r, g), b)
    min_val = np.minimum(np.minimum(r, g), b)
    diff = max_val - min_val

    h = np.zeros_like(max_val)
    s = np.zeros_like(max_val)
    v = max_val.copy()

    s = np.where(max_val != 0, diff / max_val, 0)

    mask_r = (max_val == r)
    mask_g = (max_val == g) & ~mask_r
    mask_b = (max_val == b) & ~mask_r & ~mask_g

    safe_diff = np.where(diff == 0, 1, diff)
    h[mask_r] = ((g[mask_r] - b[mask_r]) / safe_diff[mask_r]) %
6
    h[mask_g] = (b[mask_g] - r[mask_g]) / safe_diff[mask_g] + 2
    h[mask_b] = (r[mask_b] - g[mask_b]) / safe_diff[mask_b] + 4

    h = h / 6.0
    h = np.clip(h, 0, 1)

    hsv = np.stack([h, s, v], axis=2)
    return (hsv * 255).astype(np.uint8)
```

```

def nearest_neighbor_interpolate(image, new_width,
new_height):
    src_height, src_width = image.shape[:2]
    result = np.zeros((new_height, new_width, 3),
dtype=image.dtype)

    scale_y = src_height / new_height
    scale_x = src_width / new_width

    for y in range(new_height):
        for x in range(new_width):
            src_y = int((y + 0.5) * scale_y - 0.5)
            src_x = int((x + 0.5) * scale_x - 0.5)
            src_y = np.clip(src_y, 0, src_height - 1)
            src_x = np.clip(src_x, 0, src_width - 1)
            result[y, x] = image[src_y, src_x]

    return result

def bilinear_interpolate(image, new_width, new_height):
    src_height, src_width = image.shape[:2]
    result = np.zeros((new_height, new_width, 3),
dtype=np.float32)

    scale_y = (src_height - 1) / (new_height - 1) if
new_height > 1 else 0
    scale_x = (src_width - 1) / (new_width - 1) if new_width >
1 else 0

    for y in range(new_height):
        for x in range(new_width):
            src_y = y * scale_y
            src_x = x * scale_x

            y0 = int(src_y)
            x0 = int(src_x)
            y1 = min(y0 + 1, src_height - 1)
            x1 = min(x0 + 1, src_width - 1)

            fy = src_y - y0
            fx = src_x - x0

```

```

        p00 = image[y0, x0].astype(np.float32)
        p10 = image[y0, x1].astype(np.float32)
        p01 = image[y1, x0].astype(np.float32)
        p11 = image[y1, x1].astype(np.float32)

        result[y, x] = (1 - fx) * (1 - fy) * p00 + \
            fx * (1 - fy) * p10 + \
            (1 - fx) * fy * p01 + \
            fx * fy * p11

    return result.astype(np.uint8)

def scale_image(image, scale_factor, method):
    new_height = int(image.shape[0] * scale_factor)
    new_width = int(image.shape[1] * scale_factor)

    if method == "nearest":
        return nearest_neighbor_interpolate(image, new_width,
new_height)
    else:
        return bilinear_interpolate(image, new_width,
new_height)

def resize_image(image, new_width, new_height, method):
    if method == "nearest":
        return nearest_neighbor_interpolate(image, new_width,
new_height)
    else:
        return bilinear_interpolate(image, new_width,
new_height)

def rotate_image(image, angle, method):
    h, w = image.shape[:2]
    center_y, center_x = h / 2, w / 2

    angle_rad = np.radians(angle)
    cos_a = np.cos(angle_rad)
    sin_a = np.sin(angle_rad)

    new_h = int(abs(-sin_a * w) + abs(cos_a * h)) + 2
    new_w = int(abs(cos_a * w) + abs(-sin_a * h)) + 2

```

```

result = np.zeros((new_h, new_w, 3), dtype=np.uint8)
new_center_y, new_center_x = new_h / 2, new_w / 2

for y in range(new_h):
    for x in range(new_w):
        dx = x - new_center_x
        dy = y - new_center_y

        src_x = dx * cos_a + dy * sin_a + center_x
        src_y = -dx * sin_a + dy * cos_a + center_y

        src_x = np.clip(src_x, 0, w - 1)
        src_y = np.clip(src_y, 0, h - 1)

        if method == "nearest":
            result[y, x] = image[int(src_y), int(src_x)]
        else:
            x0, y0 = int(src_x), int(src_y)
            x1 = min(x0 + 1, w - 1)
            y1 = min(y0 + 1, h - 1)

            fx = src_x - x0
            fy = src_y - y0

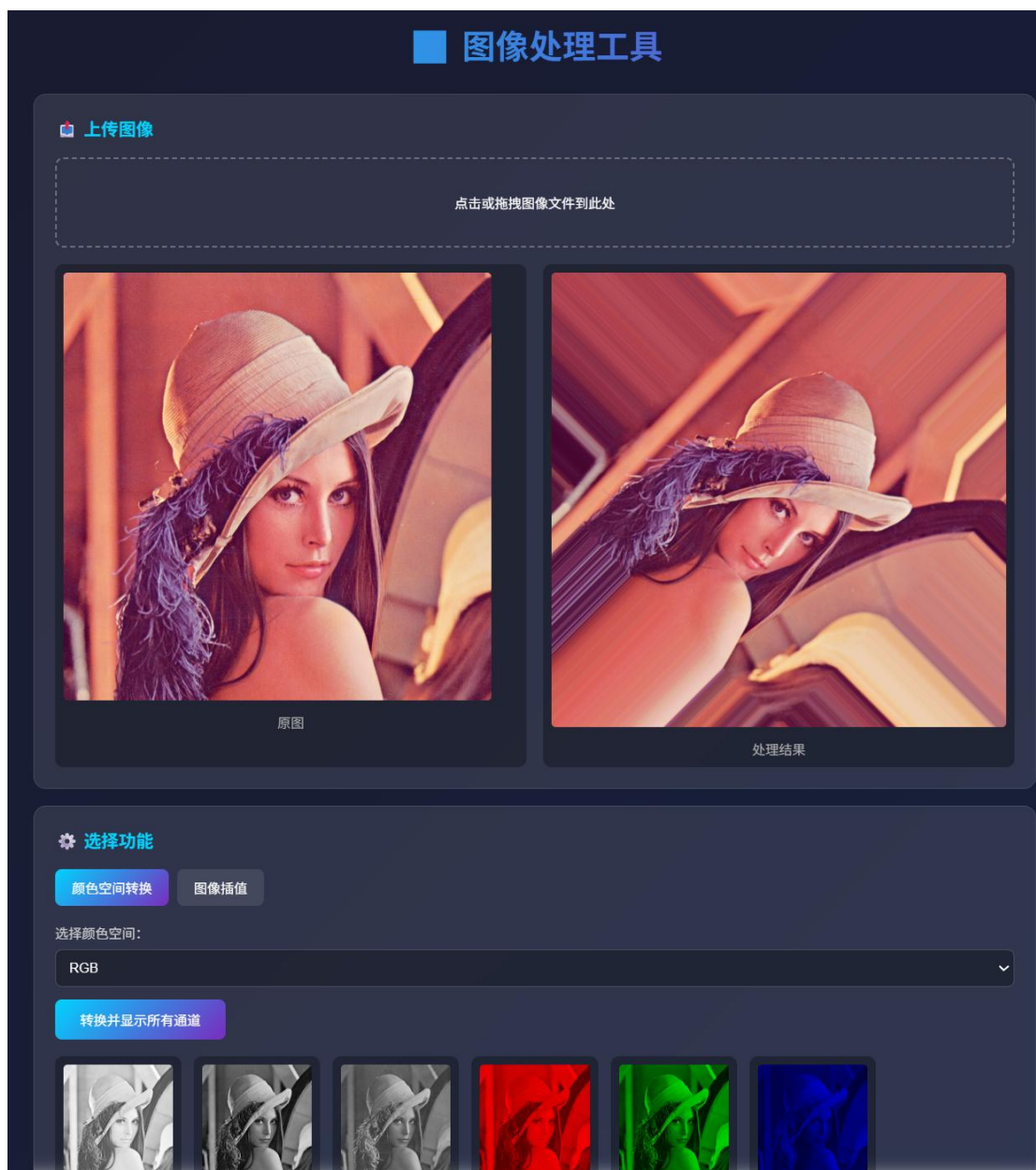
            p00 = image[y0, x0].astype(np.float32)
            p10 = image[y0, x1].astype(np.float32)
            p01 = image[y1, x0].astype(np.float32)
            p11 = image[y1, x1].astype(np.float32)

            result[y, x] = ((1 - fx) * (1 - fy) * p00 +
                             fx * (1 - fy) * p10 +
                             (1 - fx) * fy * p01 +
                             fx * fy * p11).astype(np.uint8)

    return result

```

3 测试截图



4 作业小结

理解了颜色空间的本质：RGB 是面向硬件的加色模型，适用于显示设备；HSV 是面向人眼视觉的模型，更便于色彩的语义化分析（如通过 H 通道区分不同颜色、S 通道判断色彩鲜艳程度），掌握了不同颜色空间的适用场景。

掌握了插值算法的核心逻辑：插值的本质是通过已知像素值估算未知像素值，算法的精度与计算量呈正相关，需根据实际需求选择合适的插值方式。